FlashPASTA: taking Fourier augmentation off the CPU critical path

Maryam Saif
maryam.h.saif@gmail.com

Saudi HPC/AI 2026
KAUST · 26–28 October
Methods that improve computational efficiency

Code & data

42×
FlashPASTA on 1 GPU vs official code on 32 CPU cores

77% → 0.2%
of training time the GPU sits waiting for data (8 CPU workers)

4.2×
faster YOLO training on Sim10k with the same 8 workers

3.3×10^{-6}
largest pixel difference from the official output, same noise

The problem

Synthetic frames come with free labels, but detectors trained on them lose accuracy on real photos. **PASTA** [1] closes part of the gap by randomly perturbing each image's Fourier amplitude spectrum, more strongly at high frequencies.



Sim10k frame [2]



After PASTA (α=3, β=0.25, k=2)

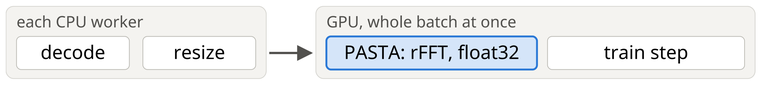
The official code runs per image inside CPU data-loader workers, with float64 noise and inverse FFT and the noise mask rebuilt on every call: **0.7 images/s per core** at 1914×1052. At 640 px it slows a worker from **77 to 5.9 images/s**.

FlashPASTA

Official



FlashPASTA



- **Batched cuFFT on the GPU** after the batch is transferred
- **Noise mask cached** per image size
- **float32 throughout**, within 3.3×10^{-6} of the official output
- **F·ε replaces $|F|\cdot\varepsilon\cdot e^{i\varphi}$**: no abs, angle or exp
- **Half-size real FFT**, exact because only the real part is kept:

```
Re IFFT(F·ε) = IRFFT(F·(ε(f)+ε(−f))/2)
```

Same output as the official code

Same noise: largest pixel difference (float32)	3.3×10^{-6}
8-bit pixels that change (by 1 level)	0.010%
Sampled noise spread vs official	0.98–1.02×
Output pixel spread vs official (2,000 draws, median)	1.00×

1 · PASTA alone: 42× faster on one GPU

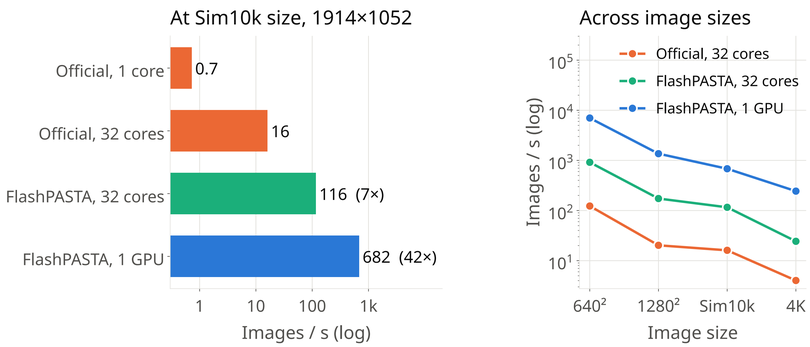


Figure 1. Augmentation only, no data loading. RTX A5000 vs Threadripper PRO 3975WX (32 cores); CPU runs use single-threaded processes, like data-loader workers.

2 · Data loading: GPU PASTA is nearly free

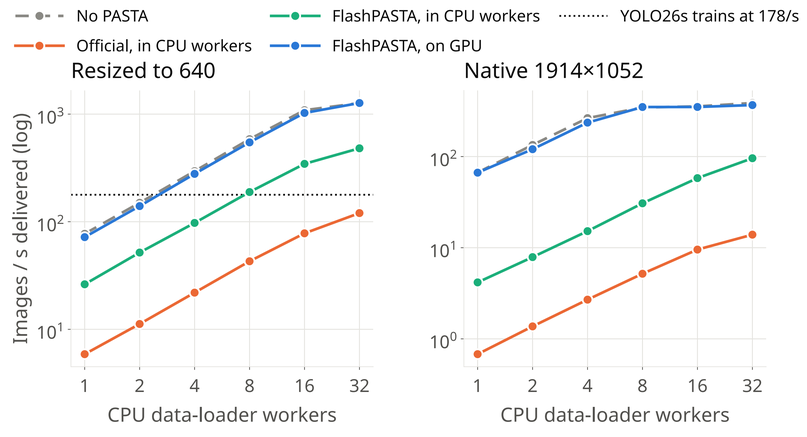


Figure 2. Images/s a PyTorch DataLoader delivers on Sim10k (JPEG decode → letterbox → PASTA) vs CPU workers.

Takeaways

- FlashPASTA on one GPU is 42× faster than the official code on 32 CPU cores, with the same output up to floating-point rounding.
- On the GPU it costs ≤1.7% of training speed with any number of workers; in CPU workers FlashPASTA needs 8 to keep up.
- Restructuring (6.4× on one core) and the real-FFT identity (1.4× on GPU) apply to any Fourier-domain augmentation.

3 · YOLO training: 4.2× faster epochs

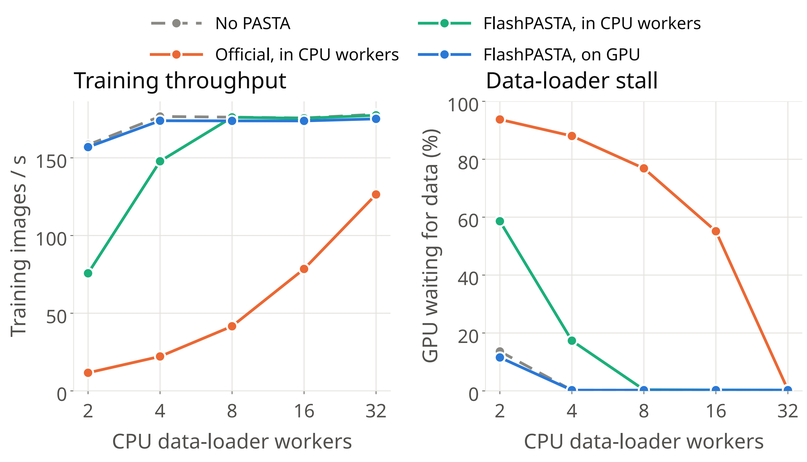


Figure 3. YOLO26s [3], 640 px, batch 16, one RTX A5000, mixed precision; mosaic off so every mode sees identical inputs.

8 CPU workers, 640 px	Images/s	GPU waiting	Sim10k epoch
Official, CPU	42	77%	4.0 min
FlashPASTA, CPU	**176**	**0.4%**	**0.9 min**
FlashPASTA, GPU	**174**	**0.2%**	**1.0 min**

The official code never keeps up with the GPU's 178 img/s: 32 workers reach 126 img/s but compete with training for the CPU, slowing each step by 41%. At 1280 px: **6.4×** (GPU vs official).

4 · Red Sea corals: 3.7× faster training

Real underwater images, Marjan dataset [4] (9,498 train), same setup.



Healthy | Bleached | Dead

8 CPU workers, 640 px	Images/s	GPU waiting	Epoch
Official, CPU	44	75%	3.6 min
FlashPASTA, CPU	**177**	**0.3%**	**0.9 min**
FlashPASTA, GPU	**166**	**0.2%**	**1.0 min**

[1] P. Chattopadhyay, K. Sarangmath, V. Vijaykumar, J. Hoffman. PASTA: Proportional Amplitude Spectrum Training Augmentation for Syn-to-Real Domain Generalization. ICCV 2023. [2] M. Johnson-Roberson et al. Driving in the Matrix. ICRA 2017. Sim10k, doi:10.7302/e1f1-3d97, CC BY-NC 4.0. [3] Ultralytics YOLO26. [4] A. Aldhahri, E. Saif, H. Ali, M. Alsayed, F. Alshareef. Harnessing Computer Vision and Deep Learning to Monitor Coral Reef Health. ETASR 15(4), 2025, doi:10.48084/etasr.10324; Marjan dataset, Roboflow Universe, CC BY 4.0.